MASDROID: A MULTI-AGENT SYSTEM FOR ENHANCING THE ANALYSIS OF ANDROID MALWARE

Xingyuan Wei[1,2], *Ziyang Yu*[1,2]*, *Lihao Liu*[1,2], *Degang Sun*[2,3], *Wentao Li*[1,2], *Yan Wang*[1,2]

[1]Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2]School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
[3]Computer Network Information Center, Chinese Academy of Sciences, Beijing, China
{weixingyuan,yuziyang,liulihao,liwentao1999,wangyan}@iie.ac.cn

ABSTRACT

With the proliferation of sophisticated Android malware, it is essential to develop detection systems that combine behavioral analysis with human-interpretable explanations. Existing approaches primarily rely on static code analysis or deep learning–based black-box models, which often overlook dynamic behavioral patterns and lack transparency in decision making. To address these limitations, we propose MASDroid, a novel multi-agent framework that integrates specialized behavioral analysis modules to provide comprehensive feature characterization of malware. The system employs context-aware agents to monitor diverse operational facets—including permission-usage patterns and function-call sequences—and leverages large language models (LLMs) to generate forensic explanations that articulate the supporting evidence in natural language. Experimental evaluations demonstrate strong performance across multiple benchmark datasets, achieving F1-scores of 97.17% for malware detection and 97.00% for malware family classification, while simultaneously improving the transparency and evidential grounding of the decision process.

Index Terms— LLM Agent, Multi-Agent, Malware Analysis, Interpretability

1. INTRODUCTION

The rapid evolution of Android malware poses substantial challenges to cybersecurity infrastructure; recent studies report that in 2023 more than 9,000 new malicious samples appeared each day [1]. Existing approaches—such as purely static or single-source feature methods—are readily evaded [2], [3]. End-to-end deep models deliver strong performance but provide limited interpretability, hindering transparency in the decision process [4]. Dynamic analysis, meanwhile, is costly and scales poorly[5]. Consequently, there is an urgent need for intelligent systems that can analyse complex behavioural patterns while providing a transparent decision process to support cybersecurity analysts.

*CORRESPONDING AUTHOR

Existing malware detection paradigms exhibit critical limitations across three dimensions:

Single-source feature extraction bias: Traditional methods often focus on isolated behavioral dimensions and overlook cross-layer synergy. For example, Drebin [2] relies exclusively on static permission patterns, whereas XMAL [6] emphasizes API call sequences. Such fragmentation leads to incomplete threat representations. MSDroid [4] exhibits a substantial drop in F1 score under reflection-based obfuscation due to its reliance on static features. Similarly, the permission–API knowledge graph in ArgusDroid [7] cannot capture runtime behavior, resulting in a 12.7% miss rate for multi-process attacks. Even state-of-the-art methods such as SNDGCN [8], under an N-hop node-retention strategy, see their FCG size reduced by 61.3%, potentially losing critical structural information. These limitations underscore the necessity of multi-dimensional feature fusion.

The opacity of the decision-making process: Black-box models dominate contemporary research, offering limited interpretability for forensic analysis. Although API2Vec [9] achieves 99.54% accuracy via behavioral paths, its Doc2Vec encoding lacks semantic interpretability. GNN-based methods such as Inferential SIR-GN [10] report an F1 score of 99.6% but yield adjacency-matrix–based decisions that are difficult to trace. Existing interpretability techniques—for example, LIME—construct only local linear approximations and may overlook high-dimensional feature interactions and decision pathways; attention mechanisms indicate "focus" but do not constitute proof of causal determinants. Prior work shows that attention weights are not equivalent to true feature importance, and overreliance on attention visualizations can be misleading [11]. Moreover, many XAI techniques emphasize local explanations and lack the capacity to summarize a model's global decision boundaries and rules [12]. Therefore, new methods are needed to enhance the transparency of the decision-making process.

To address these challenges, this study introduces MASDroid, a novel multi-agent framework that synergizes LLM capabilities with heterogeneous behavioral analysis modules. The proposed system comprises three core innovations:

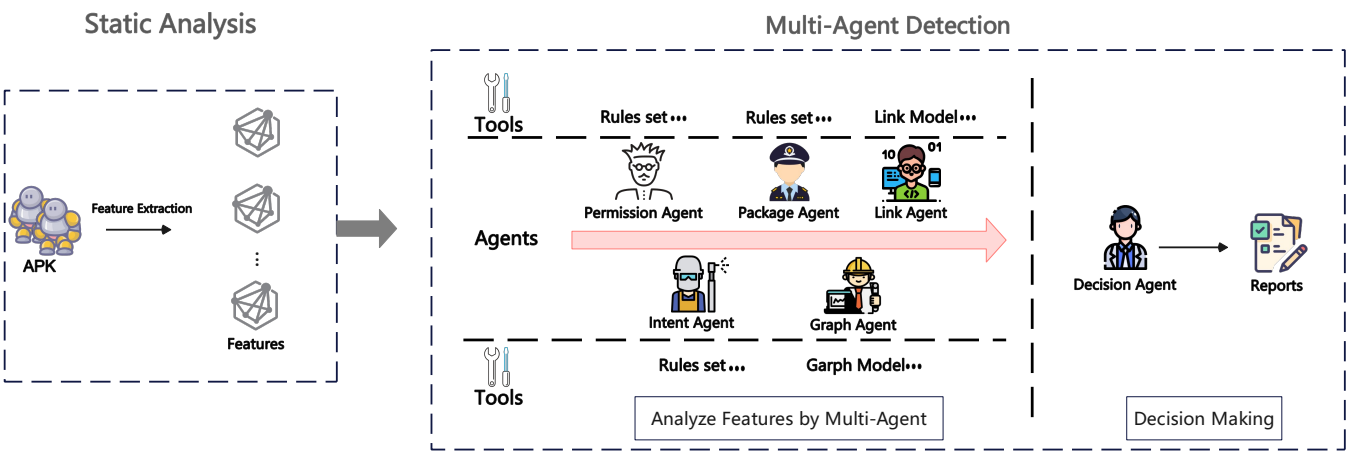

Fig. 1. Overview of MASDroid.

- We propose a novel multi-agent collaborative framework, MASDroid, to address the challenges of Android malware detection. The system comprises a Permission Agent, Link Agent, Intent Agent, Graph Data Agent, Package Agent, and a Decision Agent. By analyzing multi-dimensional features in parallel, the framework achieves more comprehensive coverage of malicious behaviors, overcoming the limitations of single-model approaches in handling feature diversity.
- Improved interpretability: Unlike black-box deep learning models, we leverage large language models (LLMs) to generate natural-language explanatory reports that detail the evidential basis of detections. This not only enhances system transparency but also provides actionable guidance for security analysts, supporting cybersecurity research.
- High performance and efficiency: Through multi-agent collaboration and LLM-based reasoning, we analyze malicious programs from static code and bytecode, avoiding the overhead and risks of runtime sandboxing. Meanwhile, the Decision Agent employs a weighted-fusion mechanism to ensure efficient and accurate classification.

2. SYSTEM DESIGN

2.1. Overall Architecture

In this paper, we propose MASDroid, an innovative multi-agent framework for malware detection. Figure 1 presents an overview of MASDroid. MASDroid consists of two components: a static analysis stage and a multi-agent detection stage. The first stage extracts feature information from malware, and the second stage analyzes these features through a multi-agent architecture.

2.2. Static Analysis

During the static analysis stage, we use Androguard[13] to parse the APK file, decompile the APK to obtain the source code, and then extract the common features. These features are subsequently distributed to all Agents. Each Agent is started in parallel without waiting for each others, ensuring that the analysis time is close to the execution time of the slowest Agent, rather than being the cumulative sum.

2.3. Agent Descriptions

Each agent is responsible for a specific task and may employ dedicated agent tools to perform the required analyses. For example, the Permission Agent uses the T5 model [14] with a learning-based approach to predict malicious behavior from text-based permission features extracted from the data, whereas the Link Agent uses a fine-tuned version of the distilbert-base-uncased model [15] to determine whether a link is suspicious. The prompts, task specifications, and all tools used by these and other agents are shown in Figure 2.

2.4. Multi-Agent Collaboration

In the MASDroid system, cooperation among agents follows a distributed, parallel, and modular architecture that enables efficient collaboration while avoiding complex interdependencies. Analogous to a multimodal fusion pipeline, the front-end agents (Permission, Link, Intent, and Graph Data) operate independently as "feature extractors and analyzers," and their outputs are then passed in a unified format to the Decision Agent, which serves as the "integrator and decision maker." The entire flow is unidirectional—from front-end to decision—to preserve the efficiency of static analysis. The overall workflow is as follows:

1. Receive an APK file as input and first perform static analysis to extract general features, which are then reused by multiple agents.

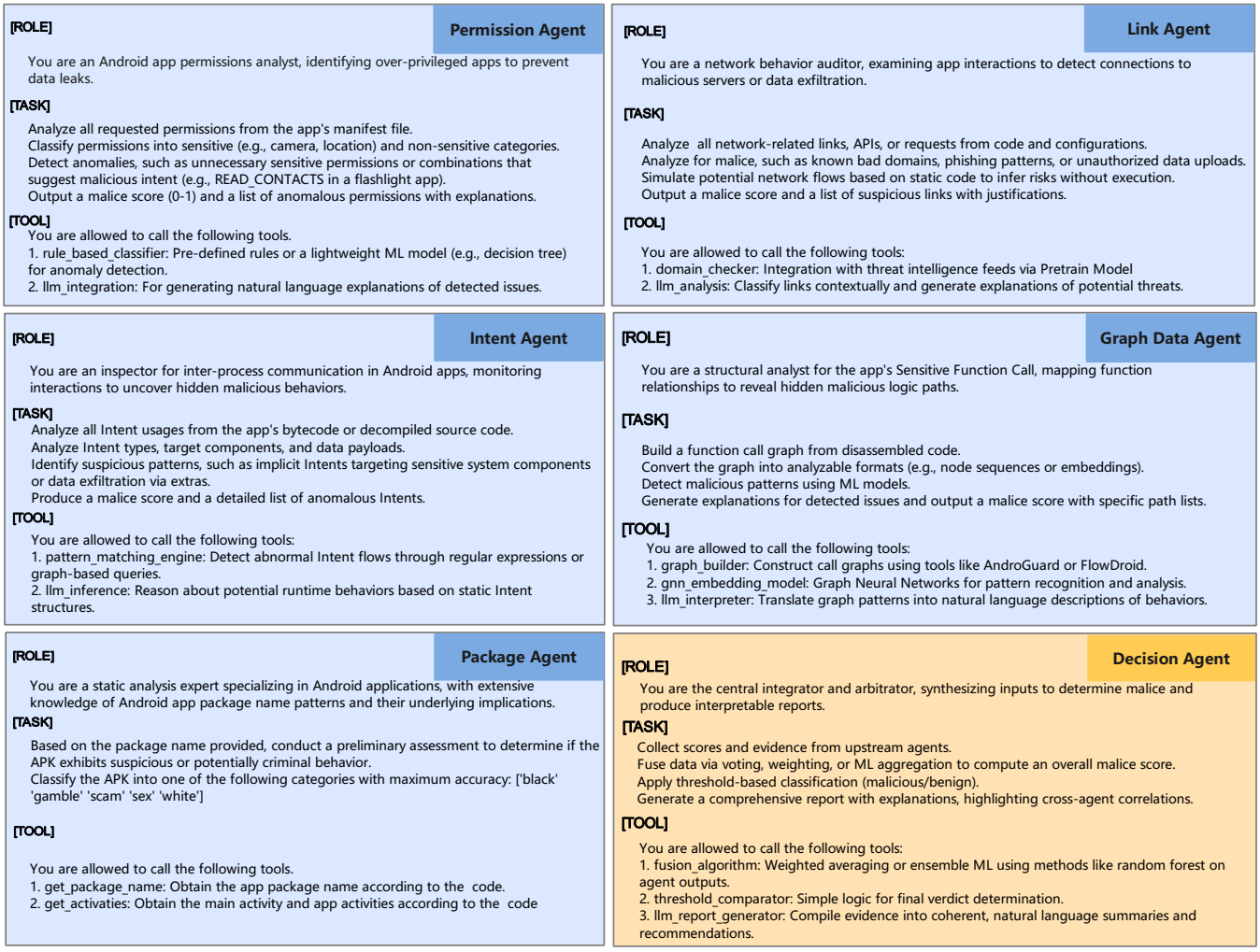

Fig. 2. All Agent tasks, system prompts and the tools available in MASDroid.

2. Each agent produces a standardized output: a maliciousness score s_i, $s_i \in [0,1]$, where 1 indicates high maliciousness, along with an evidence list e_i, which is structured data. In this study, we use a JSON list that contains specific anomalies and brief descriptions.

3. The outputs are passed to the Decision Agent, which functions as an independent module and receives an aggregated input $\mathbf{S} = [s_1, s_2, s_3, s_4, s_5]$ and $\mathbf{E} = [e_1, e_2, ..., e_5]$. This ensures that the Decision Agent has a complete multi-dimensional view without needing to backtrack to the front-end agents.

Decision Agent decision-making process: The role of the Decision Agent is to integrate multi-source information, perform the final classification, and generate an explanatory report. We adopt a weighted-fusion approach with three stages: score fusion, thresholding, and report generation.

In the score-fusion stage, we compute the overall maliciousness score S_{total} using a weighted average. The weights $\mathbf{W} = [w_1, w_2, w_3, w_4, w_5]$, $w_i \in [0,1]$ reflect the importance of each Agent, and in this work the weights are preset. The fusion formula is given in 1, where $n = 5$ denotes the number of Agents. $s_i \in [0,1]$ denotes the maliciousness score of the i-th Agent. $w_i \in [0,1]$, $\sum_{i=1}^{n} w_i = 1$, denotes the weights. $\theta \in [0,1]$ is the decision threshold. $c_{ij} \in [0,1]$ is the evidence correlation between Agents i and j. α is the correlation amplification factor, set to 0.05 in this paper.

$$S_{total} = \sum_{i=1}^{n} w_i s_i + \alpha \sum_{i<j} c_{ij}(s_i + s_j) \tag{1}$$

In the thresholding stage, we set the decision threshold θ. If $S_{total} \geq \theta$, the sample is classified as malicious; otherwise, it is classified as benign. The formulation is given in 2.

Table I. Performance comparison of the proposed models with deep learning models (%)

Method		F1-Score	FPR	FNR
NATICUSdroid[3]	SVM	95.07	3.12	5.24
	Random Forest	94.81	3.80	4.82
	XGBoost	95.01	3.60	4.71
FCG[18]	GCN	59.98	5.80	53.60
	GraphSAGE	60.19	5.88	53.31
	TAGCN	59.20	5.83	55.50
ResNet[19]	ResNet-18	86.90	3.49	11.95
	ResNet-50	85.02	3.70	14.60
Ours	deepseek-r1:32b	**97.17**	**2.10**	**3.36**
	qwq:32b	**97.12**	**2.32**	**3.10**
	qwen3:32b	**96.19**	**3.30**	**3.19**

Table II. Family classification results(%)

Method		F1-Score	FPR	FNR
GNN[18]	Null Feature	88.82	8.90	13.21
	Structural Feature	94.50	5.29	7.43
Ours	deepseek-r1:32b	**96.19**	**2.30**	**5.30**
	qwq:32b	**96.62**	**2.61**	**4.20**
	qwen3:32b	**97.00**	**2.00**	**4.10**

$$\text{Classification} = \begin{cases} \text{Malicious} & \text{if } S_{total} \geq \theta \\ \text{Benign} & \text{otherwise} \end{cases} \tag{2}$$

In the report generation stage, an LLM produces a natural-language report based on $\mathbf{E}$ and S_{total}. The report includes an overview, evidence organized by agent (e.g., "the Permission Agent's anomalous permissions and the Graph Data Agent's call paths"), and mitigation recommendations.

3. EVALUATION

3.1. Dataset & Metrics

Our data sources are AndroZoo [16] and CICDataset-2020 [17]. From AndroZoo, we extracted 11,500 benign samples and 11,000 malicious samples to construct our detection dataset. We use CICDataset-2020 [17] for the family-classification task. To evaluate the performance of our multi-agent framework, we adopt the following key metrics: F1 score, FNR, and FPR.

3.2. Malware Detection And Familiar Classification

For the binary classification task, Table I shows that our method overall outperforms traditional machine learning, graph neural networks, and vision-based baselines. Specifically, deepseek-r1:32b achieves the highest F1 = 97.17% and reduces FPR to 2.10%, while qwq:32b attains the lowest FNR at 3.10%. Compared with the best traditional baseline, our approach yields significant gains in accuracy and missed-detection control; by contrast, the FCG family suffers from markedly lower F1 due to an excessively high FNR. The three backbones maintain a consistently favorable balance between FPR and FNR, indicating that the framework adapts well to different semantic modeling capacities. The clear gap from ResNet-18/50 further suggests that, in static-analysis settings, a paradigm that fuses structural and semantic evidence is more practically valuable.

For the family classification task, Table II indicates that structured graph features outperform empty features, and our method further improves performance into the 96–97% range. In particular, qwen3:32b achieves the best overall performance, simultaneously reducing both false positives and false negatives relative to GNN (Structural Feature), demonstrating a stable advantage in fine-grained family discrimination. Compared with GNNs that rely solely on graph structure, the gains are most pronounced for boundary classes and closely related families, highlighting the complementarity of multi-source evidence fusion and semantic reasoning.

3.3. Result Report

As shown in Figure 3, MASDroid's partial report is a structured JSON output containing the total malice score, classification, confidence, per-agent scores, and brief summaries. It provides quantitative indicators and preliminary evidence; however, current interpretability has limitations, such as insufficient modeling of logical relationships among pieces of evidence and limited impact analysis of potential risks. For more detailed reports and codes, you can obtain them from GitHub[1].


```
{
  "sha256": "00261026DA5A6D169966AA2528B5E96838433979DC543325C9A802E8AF0F0329",
  "total_malice_score": 0.92,
  "classification": "Malicious",
  "confidence": 0.7927007928469842,
  "agent_scores": {
    "permission": 0.1917808219178082,
    "activity": 0.0,
    "package": 0.42,
    "link": 1.0
  ...
  },
  "summaries": {
    "permission": "Analysis complete. Found 7 dangerous permissions.",
    "activity": "Activity analysis complete. No immediate suspicious indicators found.",
    "package": "Package analysis complete. Potential risks detected.",
    "link": "Early exit triggered (rule_hits_reached_4). Found 4 suspicious or malicious links (potential C&C or downloads)."
  }
```


Fig. 3. Result Output Report file.

4. CONCLUSION

MASDroid standardizes multi-source static features into scores and evidence sets, and uses weighted fusion to produce the final decision and an interpretable report. Experiments show consistent gains over traditional machine learning, GNNs, and vision baselines on both binary and family classification, while substantially reducing FPR and FNR, validating the effectiveness of the multi-agent design. Future work will incorporate lightweight dynamic behavior, introduce learnable weights and uncertainty modeling, and refine cross-agent evidence alignment to further enhance robustness and practicality.

[1] https://github.com/XingYuanWei/MASDroid

5. REFERENCES

[1] AV-TEST Institute, "Mobile malware report q1 2024," Tech. Rep., 2024.

[2] Daniel Arp, Michael Spreitzenbarth, Malte Hubner, Hugo Gascon, and Konrad Rieck, "Drebin: Effective and explainable detection of android malware in your pocket," *NDSS*, 2014.

[3] Akshay Mathur, Laxmi Mounika Podila, Keyur Kulkarni, Quamar Niyaz, and Ahmad Y. Javaid, "Naticusdroid: A malware detection framework for android using native and custom permissions," *Journal of Information Security and Applications*, vol. 58, pp. 102696, 2021.

[4] Yiling He, Yiping Liu, Lei Wu, Ziqi Yang, Kui Ren, and Zhan Qin, "Msdroid: Identifying malicious snippets for android malware detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 3, pp. 2025–2039, 2023.

[5] Ce Li, Zijun Cheng, He Zhu, Leiqi Wang, Qiujian Lv, Yan Wang, Ning Li, and Degang Sun, "Dmalnet: Dynamic malware analysis based on api feature engineering and graph learning," *Computers Security*, vol. 122, pp. 102872, 2022.

[6] Bozhi Wu, Sen Chen, Cuiyun Gao, Lingling Fan, Yang Liu, Weiping Wen, and Michael R. Lyu, "Why an android app is classified as malware? towards malware classification interpretation," *ACM Transactions on Software Engineering and Methodology*, 2021.

[7] Yude Bai, Sen Chen, Zhenchang Xing, and Xiaohong Li, "ArgusDroid: Detecting Android malware variants by mining permission-API knowledge graph," vol. 66, no. 9, pp. 192101.

[8] Xiaofeng Lu, Jinglun Zhao, Senhao Zhu, and Pietro Lio, "SNDGCN: Robust Android malware detection based on subgraph network and denoising GCN network," vol. 250, pp. 123922.

[9] Lei Cui, Jiancong Cui, Yuede Ji, Zhiyu Hao, Lun Li, and Zhenquan Ding, "API2Vec: Learning Representations of API Sequences for Malware Detection," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. pp. 261–273, ACM.

[10] Jeffrey Fairbanks, Janet Layne, Andres Orbe, Edoardo Serra, Christine Patterson, and Marion Scheepers, "Identifying att&ck tactics in android malware control flow graph through graph representation learning and interpretability," in *IEEE European Symposium on Security and Privacy Workshops*, 2021.

[11] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin, ""why should i trust you?": Explaining the predictions of any classifier," 2016.

[12] Harikha Manthena, Shaghayegh Shajarian, Jeffrey C. Kimmell, Mahmoud Abdelsalam, Sajad Khorsandroo, and Maanak Gupta, "Explainable artificial intelligence (xai) for malware analysis: A survey of techniques, applications, and open challenges," *IEEE Access*, vol. 13, pp. 61611–61640, 2025.

[13] "GitHub - androguard/androguard: Reverse engineering and pentesting for Android applications — github.com," https://github.com/androguard/androguard, [Accessed 07-06-2024].

[14] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, Jan. 2020.

[15] "kmack/malicious-url-detection · Hugging Face — huggingface.co," https://huggingface.co/kmack/malicious-url-detection, [Accessed 17-09-2025].

[16] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon, "Androzoo: Collecting millions of android apps for the research community," in *Proceedings of the 13th International Conference on Mining Software Repositories*, New York, NY, USA, 2016, MSR '16, pp. 468–471, ACM.

[17] Samaneh Mahdavifar, Dima Alhadidi, and Ali. A. Ghorbani, "Effective and efficient hybrid android malware classification using pseudo-label stacked auto-encoder," *J. Netw. Syst. Manage.*, vol. 30, no. 1, Jan. 2022.

[18] Vinayaka K V and Jaidhar C D, "Android malware detection using function call graph with graph convolutional networks," in *2021 2nd International Conference on Secure Cyber Computing and Communications (ICSCCC)*, 2021, pp. 279–287.

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015.